

JAVASCRIPT PROGRAMMING LAB (BCS354)

B.Tech CSE - 2nd SEMESTER

SUBMITTED TO
MR. PRADEEP SINGH

SUBMITTED BY
Shubhankar Singh
ID: BCS2025390

INDEX

Name: SHUBHANKAR SINGH **Roll No: BCS2025390**
Section: "E"

S.No.	Name of the Experiment	Page	Date	Sign
1	Your First JavaScript Script	3		
2	Operators, Type Conversion and a Calculator	5		
3	Conditions and Loops	8		
4	Writing Functions	11		
5	Arrays, Array Methods and Nested Arrays	13		
6	JSON Text	16		
7	Constructor Functions and Classes	18		
8	Inheritance, Methods and Composition	21		
9	DOM, Validation, Storage and Quiz	24		
10	Objects and Destructuring	29		
11	Scope and Closures	31		
12	Callbacks and Higher-Order Functions	34		
13	Reduce, Some, Every and Map	37		
14	Timers and Intervals	39		
15	Promises and async/await with Local Data	42		

EXPERIMENT NO. 1

Date of Performance:- Course: BCS354

- **Experiment Title:**

Your First JavaScript Script

2. Aim:

- To run JavaScript in a browser and with Node.js, and to declare values using const and let.

3. Software / Tools Required:

Visual Studio Code, Node.js, Google Chrome

4. Theory:

- JavaScript is a programming language used to make web pages interactive. The console.log() function shows a message in the console. The const keyword creates a value that cannot be changed, while let creates a variable whose value can be changed. The typeof operator tells the data type of a value, such as string, number or boolean.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: Print a message in the browser

6. Program:

```
<!DOCTYPE html>
<!-- Name: Shubhankar Singh | ID: BCS2025390 | Assignment 1 | Task 1 -->
<html>
<head><title>First Script</title></head>
<body>
  <h1>First JavaScript Page</h1>
  <script>
    console.log("Hello from the script");
  </script>
</body>
</html>
```

7. Input:

- Open the HTML file in the browser and press F12 to open the Console.

8. Output:

```
Console
Hello from the script                                    script.html:8
```

Task 2: const and let

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 1 | Task 2
const collegeName = "Invertis University";
let year = 2026;

console.log(collegeName);
console.log(year);
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment1\Task2> node
task2.js
Invertis University
2026
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment1\Task2>
```

Task 3: typeof

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 1 | Task 3
console.log(typeof "hello");
console.log(typeof 7);
console.log(typeof true);
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment1\Task3> node
task3.js
string
number
boolean
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment1\Task3>
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the difference between const and let?

Ans. A const value cannot be changed after it is set, but a let variable can be changed.

Q2. What does console.log() do?

Ans. It prints a message or value in the console.

Q3. What does typeof return?

Ans. It returns the data type of a value, as text.

Q4. Where do you see the console output of a web page?

Ans. In the browser Console, opened with the F12 key.

EXPERIMENT NO. 2

Date of Performance:- Course: BCS354

1. Experiment Title:

- Operators, Type Conversion and a Calculator

2. Aim:

- To use arithmetic operators, to convert between text and numbers, and to use if and else if to choose an operation.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- Arithmetic operators such as +, -, * and / are used to calculate with numbers. Number() converts text to a number and String() converts a number to text. The + operator adds two numbers but joins two pieces of text. The if and else if statements choose which block of code to run depending on a condition, for example the operator chosen in a calculator.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: Arithmetic operators

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 2 | Task 1
const a = 17;
const b = 5;

console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a / b);
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment2\Task1> node
task1.js
22
12
85
3.4
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment2\Task1>
```

Task 2: Type conversion

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 2 | Task 2
const num = Number("12");
console.log(num + 5);

const text = String(7);
console.log(text + "3");
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment2\Task2> node
task2.js
17
73
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment2\Task2>
```

Task 3: Simple calculator with if and else if

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 2 | Task 3
const a = 8;
const b = 4;
let operator = "+";

function check() {
  if (operator === "+") {
    console.log("Sum: " + (a + b));
  } else if (operator === "-") {
    console.log("Difference: " + (a - b));
  }
}

check();
operator = "-";
check();
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment2\Task3> node
task3.js
Sum: 12
Difference: 4
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment2\Task3>
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. Why does "7" + "3" give "73"?

Ans. With text, the + operator joins the values instead of adding them.

Q2. What does Number("12") do?

Ans. It converts the text 12 into the number 12.

Q3. What is the difference between == and ===?

Ans. === checks both value and type, while == changes the type before comparing.

Q4. What is the result of 17 / 5?

Ans. It is 3.4, because division in JavaScript gives a decimal result.

EXPERIMENT NO. 3

Date of Performance:- Course: BCS354

1. Experiment Title:

- Conditions and Loops

2. Aim:

- To choose between cases with if, else if and else, and to repeat work with a for loop and a while loop.

3. Software / Tools Required:

Visual Studio Code, Node.js, Google Chrome

4. Theory:

- The if, else if and else statements choose one block of code depending on a condition. A for loop repeats code a fixed number of times, and a while loop repeats code as long as its condition is true. In a while loop the variables must change inside the loop, otherwise the loop never ends.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: Grade using if, else if and else

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 3 | Task 1
const marks = 72;
if (marks >= 75) {
  console.log("Grade A");
} else if (marks >= 60) {
  console.log("Grade B");
} else {
  console.log("Grade C");
}
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment3\Task1> node
task1.js
Grade B
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment3\Task1>
```

Task 2: for loop

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 3 | Task 2
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment3\Task2> node
task2.js
1
2
3
4
5
```

Task 3: while loop

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 3 | Task 3
let total = 0;
let n = 1;
while (total <= 20) {
  total = total + n;
  n++;
}
console.log(total);
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment3\Task3> node
task3.js
21
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the difference between a for loop and a while loop?

Ans. A for loop is used when the number of repeats is known, and a while loop runs as long as its condition is true.

Q2. What are the three parts of a for loop?

Ans. The starting value, the condition and the update step.

Q3. What happens if the condition of a while loop never becomes false?

Ans. The loop never ends (an infinite loop).

Q4. Why does the order of conditions matter in else if?
Ans. The first true condition runs and the rest are skipped.

EXPERIMENT NO. 4

Date of Performance:- Course: BCS354

1. Experiment Title:

- Writing Functions

2. Aim:

- To write a function with a return value, an arrow function and a default parameter.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- A function is a reusable block of code. It takes values through parameters and sends a result back with return. An arrow function is a shorter way of writing a function. A default parameter gives a parameter a value when the caller does not pass one.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: Function with a return value

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 4 | Task 1
function greet(name) {
  return "Hello, " + name;
}

console.log(greet("Asha"));
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment4\Task1> node
task1.js
Hello, Asha
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment4\Task1>
```

Task 2: Arrow function

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 4 | Task 2
const square = (x) => x * x;

console.log(square(6));
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment4\Task2> node
task2.js
36
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment4\Task2>
```

Task 3: Default parameter

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 4 | Task 3
function billWithTax(amount, taxRate = 0.18) {
  return amount + amount * taxRate;
}

console.log(billWithTax(100));
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment4\Task3> node
task3.js
118
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment4\Task3>
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What does the return statement do?

Ans. It sends a value back to the place where the function was called.

Q2. What is an arrow function?

Ans. It is a shorter way of writing a function, for example (x) => x * x.

Q3. What is a default parameter?

Ans. It is a value used for a parameter when no value is passed.

Q4. What is the difference between a parameter and an argument?

Ans. A parameter is written in the function definition, an argument is the value passed when calling it.

EXPERIMENT NO. 5

Date of Performance:- Course: BCS354

1. Experiment Title:

- Arrays, Array Methods and Nested Arrays

2. Aim:

- To create an array, read its items, use forEach and filter, and read a value from an array inside an array.

3. Software / Tools Required:

Visual Studio Code, Node.js, Google Chrome

4. Theory:

- An array stores many values in one variable, and each item is read by its index, which starts from 0. The forEach() method runs a function for every item, and filter() makes a new array with only the items that pass a test. An array can also hold other arrays, so grid[1][2] reads row 2, column 3.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: Array items and length

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 5 | Task 1
const marks = [72, 85, 64, 90];

console.log(marks.length);
console.log(marks[0]);
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment5\Task1> node
task1.js
4
72
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment5\Task1>
```

Task 2: forEach

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 5 | Task 2
const subjects = ["Maths", "Science", "English"];

subjects.forEach((subject) => {
  console.log(subject);
});
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment5\Task2> node
task2.js
Maths
Science
English
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment5\Task2>
```

Task 3: filter

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 5 | Task 3
const marks = [72, 85, 45, 90];
const passed = marks.filter((mark) => mark >= 60);

console.log(passed);
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment5\Task3> node
task3.js
[ 72, 85, 90 ]
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment5\Task3>
```

Task 4: Array inside an array

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 5 | Task 4
const grid = [[1, 2, 3], [4, 5, 6]];

console.log(grid[1][2]);
```

7. Input:

- Run the file using Node.js: node task4.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment5\Task4> node
task4.js
6
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment5\Task4>
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the index of the first item of an array?

Ans. It is 0.

Q2. What is the difference between forEach() and filter()?

Ans. forEach() runs code for every item, while filter() returns a new array of the items that pass a test.

Q3. How do you find the number of items in an array?

Ans. With the length property, for example marks.length.

Q4. How do you read a value from an array inside an array?

Ans. Use two indexes, grid[row][column], both starting from 0.

EXPERIMENT NO. 6

Date of Performance:- Course: BCS354

1. Experiment Title:

- JSON Text

2. Aim:

- To convert an object to JSON text, to convert JSON text back to an object, and to catch invalid JSON.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- JSON is a text format used to store and send data. JSON.stringify() converts an object into JSON text, and JSON.parse() converts JSON text back into an object. If the text is not valid JSON, JSON.parse() gives an error, which can be handled with try and catch.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: JSON.stringify

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 6 | Task 1
const student = { name: "Rohan", marks: [70, 80] };
const text = JSON.stringify(student);

console.log(text);
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment6\Task1> node
task1.js
{"name":"Rohan","marks":[70,80]}
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment6\Task1>
```

Task 2: JSON.parse

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 6 | Task 2
const text = '{"city":"Pune","pin":411001}';
const obj = JSON.parse(text);

console.log(obj.city);
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment6\Task2> node
task2.js
Pune
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment6\Task2>
```

Task 3: Invalid JSON with try and catch

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 6 | Task 3
const text = "{name: Rohan}";

try {
  JSON.parse(text);
} catch (error) {
  console.log("Invalid JSON");
}
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment6\Task3> node
task3.js
Invalid JSON
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment6\Task3>
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the difference between JSON.stringify() and JSON.parse()?

Ans. stringify() converts an object to JSON text, and parse() converts JSON text to an object.

Q2. Why is {name: Rohan} not valid JSON?

Ans. In JSON the keys and text values must be written in double quotes.

Q3. What is the use of try and catch?

Ans. It catches an error so that the program does not stop.

Q4. Where is JSON used?

Ans. To store data and to send data between a server and a browser.

EXPERIMENT NO. 7

Date of Performance:- Course: BCS354

1. Experiment Title:

- Constructor Functions and Classes

2. Aim:

- To create objects with a constructor function and with an ES6 class, and to add a method through the prototype.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- A constructor function is used with the new keyword to create objects, and this refers to the new object. An ES6 class does the same job with a cleaner syntax and a constructor() method. A method added to the prototype is shared by all objects created from that constructor.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: Constructor function

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 7 | Task 1
function Student(name, marks) {
  this.name = name;
  this.marks = marks;
}

const s = new Student("Asha", 88);
console.log(s.name);
console.log(s.marks);
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment7\Task1> node
task1.js
Asha
88
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment7\Task1>
```

Task 2: ES6 class

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 7 | Task 2
class Book {
  constructor(title, pages) {
    this.title = title;
    this.pages = pages;
  }

  summary() {
    return this.title + " has " + this.pages + " pages";
  }
}

const book = new Book("Algorithms", 320);
console.log(book.summary());
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment7\Task2> node
task2.js
Algorithms has 320 pages
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment7\Task2>
```

Task 3: Method on the prototype

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 7 | Task 3
function Student(name, marks) {
  this.name = name;
  this.marks = marks;
}

Student.prototype.result = function () {
  return this.marks >= 40 ? "Pass" : "Fail";
};

const s = new Student("Asha", 88);
console.log(s.result());
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment7\Task3> node
task3.js
Pass
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment7\Task3>
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What does the new keyword do?

Ans. It creates a new object and runs the constructor.

Q2. What does this refer to inside a constructor?

Ans. It refers to the new object that is being created.

Q3. Why do we add methods to the prototype?

Ans. The method is shared by all objects instead of being copied into each one.

Q4. What is the difference between a constructor function and a class?

Ans. A class is a newer and cleaner way to write the same idea.

EXPERIMENT NO. 8

Date of Performance:- Course: BCS354

1. Experiment Title:

- Inheritance, Methods and Composition

2. Aim:

- To use extends to make a child class, to write a method that checks a value, and to place one object inside another.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- Inheritance lets a child class reuse the properties and methods of a parent class using the extends keyword, and super() calls the parent constructor. A method can check a value and return true or false. Composition means one object holds another object, like a Car that has an Engine.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: Inheritance with extends

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 8 | Task 1
class Person {
  constructor(name) {
    this.name = name;
  }

  greet() {
    return "Hi, I am " + this.name;
  }
}

class Teacher extends Person {
  constructor(name, subject) {
    super(name);
    this.subject = subject;
  }
}

const t = new Teacher("Anita", "Maths");
console.log(t.greet());
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment8\Task1> node
task1.js
Hi, I am Anita
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment8\Task1>
```

Task 2: Method that checks a value

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 8 | Task 2
class Locker {
  constructor() {
    this.pin = 1234;
  }

  checkPin(number) {
    return this.pin === number;
  }
}

const locker = new Locker();
console.log(locker.checkPin(1111));
console.log(locker.checkPin(1234));
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment8\Task2> node
task2.js
false
true
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment8\Task2>
```

Task 3: Composition: Car and Engine

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 8 | Task 3
class Engine {
  constructor(power) {
    this.power = power;
  }
}

class Car {
  constructor(power) {
    this.engine = new Engine(power);
  }

  start() {
    return "Car starts with " + this.engine.power + " HP";
  }
}

const car = new Car(150);
console.log(car.start());
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment8\Task3> node
task3.js
Car starts with 150 HP
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment8\Task3>
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is inheritance?

Ans. It lets a child class use the properties and methods of a parent class.

Q2. Why do we call super() in a child constructor?

Ans. It runs the parent constructor so the parent properties are set.

Q3. What is the difference between inheritance and composition?

Ans. Inheritance is an is-a relation (Teacher is a Person), composition is a has-a relation (Car has an Engine).

Q4. What does checkPin return?

Ans. It returns true if the number equals the pin, otherwise false.

EXPERIMENT NO. 9

Date of Performance:- Course: BCS354

1. Experiment Title:

- DOM, Validation, Storage and Quiz

2. Aim:

- To change a page with a click, to check form values in code, to save text in local storage, and to build a small quiz.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- The DOM shows the web page as objects, so JavaScript can find an element and change its text when an event such as a click happens. A validation function checks the values entered by the user and returns a list of errors. The localStorage keeps text in the browser even after the page is closed, and a quiz compares the selected radio button with the correct answer.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.**Task 1: Change text on click**

6. Program:

```
<!DOCTYPE html>
<!-- Name: Shubhankar Singh | ID: BCS2025390 | Assignment 9 | Task 1 -->
<html>
<head>
  <title>Assignment 9 - Task 1</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 20px; }
    button { padding: 6px 14px; font-size: 15px; }
  </style>
</head>
<body>
  <h1 id="heading">Welcome</h1>
  <button id="btn">Change text</button>
  <script>
    const heading = document.getElementById("heading");
    const btn = document.getElementById("btn");

    btn.addEventListener("click", function () {
      heading.textContent = "Button clicked";
    });
  </script>
</body>
</html>
```

7. Input:

- Open the HTML file in the browser and click the Change text button.

8. Output:

Before clicking the button

```
Welcome
[ Change text ]
```

After clicking the button

```
Button clicked
[ Change text ]
```

Task 2: Form validation function

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 9 | Task 2
function validateForm(name) {
  const errors = [];
  if (name.trim() === "") {
    errors.push("Name is required");
  }
  return errors;
}

console.log(validateForm("Asha"));
console.log(validateForm(""));
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment9\Task2> node
task2.js
[]
[ 'Name is required' ]
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment9\Task2>
```

Task 3: Save text in localStorage

6. Program:

```
<!DOCTYPE html>
<!-- Name: Shubhankar Singh | ID: BCS2025390 | Assignment 9 | Task 3 -->
<html>
<head>
  <title>Assignment 9 - Task 3</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 20px; }
    input, button { padding: 6px 10px; font-size: 15px; }
  </style>
</head>
<body>
  <input type="text" id="note" placeholder="Write a note">
  <button id="save">Save</button>
  <p id="msg"></p>

  <script>
    const box = document.getElementById("note");
    const msg = document.getElementById("msg");

    // When the page opens, show the saved text
    box.value = localStorage.getItem("note") || "";

    document.getElementById("save").addEventListener("click", function () {
      localStorage.setItem("note", box.value);
      msg.textContent = "Saved";
    });
  </script>
</body>
</html>
```

7. Input:

- Open the HTML file in the browser, type some text, click Save and then reload the page.

8. Output:

Page opened for the first time (box is empty)

```
[                    ] [ Save ]
(note box is empty)
```

Text typed and Save clicked

```
[ Buy a Java book    ] [ Save ]
Saved
```

Page opened again - saved text is shown

```
[ Buy a Java book    ] [ Save ]
(saved text shown after reload)
```

Task 4: Small quiz

6. Program:

```
<!DOCTYPE html>
<!-- Name: Shubhankar Singh | ID: BCS2025390 | Assignment 9 | Task 4 -->
<html>
<head>
  <title>Assignment 9 - Task 4</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 20px; }
    button { padding: 6px 14px; font-size: 15px; margin-top: 8px; }
  </style>
</head>
<body>
  <h3>What is 2 + 3?</h3>
  <label><input type="radio" name="ans" value="4"> 4</label><br>
  <label><input type="radio" name="ans" value="5"> 5</label><br>
  <label><input type="radio" name="ans" value="6"> 6</label><br>
  <button id="check">Check answer</button>
  <p id="result"></p>

  <script>
    document.getElementById("check").addEventListener("click", function () {
      const chosen = document.querySelector("input[name='ans']:checked");
      const result = document.getElementById("result");
      if (chosen && chosen.value === "5") {
        result.textContent = "Correct";
      } else {
        result.textContent = "Try again";
      }
    });
  </script>
</body>
</html>
```

7. Input:

- Open the HTML file in the browser, choose an answer and click Check answer.

8. Output:

Answer 5 selected - Check answer

```
What is 2 + 3?
( ) 4
(o) 5
( ) 6
[ Check answer ]
Correct
```

Answer 4 selected - Check answer

```
What is 2 + 3?
(o) 4
( ) 5
( ) 6
[ Check answer ]
Try again
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the DOM?

Ans. It is the page shown to JavaScript as objects that can be found and changed.

Q2. What does addEventListener() do?

Ans. It runs a function when an event such as a click happens.

Q3. How is localStorage different from a normal variable?

Ans. A variable is lost when the page is closed, but localStorage keeps the text in the browser.

Q4. How do you find the selected radio button?

Ans. With querySelector and the :checked selector.

EXPERIMENT NO. 10

Date of Performance:- Course: BCS354

1. Experiment Title:

- Objects and Destructuring

2. Aim:

- To create an object, read and change its properties, and take values out of an array and an object with destructuring.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- An object stores related data as properties, which are read with the dot notation and changed by giving a new value. Array destructuring takes values out of an array by position, and object destructuring takes values out of an object by property name. Destructuring creates several variables in one statement.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: Object properties

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 10 | Task 1
const student = { name: "Rohan", roll: 12, marks: 78 };

console.log(student.name);
student.marks = 81;
console.log(student.marks);
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment10\Task1> node
task1.js
Rohan
81
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment10\Task1>
```

Task 2: Array destructuring

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 10 | Task 2
const subjects = ["Maths", "Science", "English"];
const [first, second] = subjects;

console.log(first);
console.log(second);
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment10\Task2> node
task2.js
Maths
Science
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment10\Task2>
```

Task 3: Object destructuring

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 10 | Task 3
const student = { name: "Rohan", roll: 12, marks: 81 };
const { name, roll } = student;

console.log(name);
console.log(roll);
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment10\Task3> node
task3.js
Rohan
12
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment10\Task3>
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. How do you read a property of an object?

Ans. With the dot notation, for example student.name.

Q2. What is the difference between array and object destructuring?

Ans. Array destructuring uses the position, object destructuring uses the property name.

Q3. Can the properties of a const object be changed?

Ans. Yes, the properties can change, but the variable cannot be assigned a new object.

Q4. What does const [first, second] = subjects do?

Ans. It puts the first two items of the array into first and second.

EXPERIMENT NO. 11

Date of Performance:- Course: BCS354

1. Experiment Title:

- Scope and Closures

2. Aim:

- To explain block scope and global scope, and to use a function that returns a function which keeps a value between calls.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- Scope decides where a variable can be used. A let or const variable declared inside a block { } can be used only inside that block, while a global variable can be used everywhere. A closure is a function that remembers the variables of the place where it was created, so a counter can keep its value between calls.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: Block scope

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 11 | Task 1
let level = "outer";

{
  let level = "inner";
  console.log(level);
}

console.log(level);
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment11\Task1> node
task1.js
inner
outer
```

Task 2: Closure counter

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 11 | Task 2
function makeCounter() {
  let count = 0;
  return function () {
    count = count + 1;
    return count;
  };
}
const counter = makeCounter();
console.log(counter());
console.log(counter());
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment11\Task2> node
task2.js
1
2
```

Task 3: Global scope

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 11 | Task 3
const school = "Sample College";
function showSchool() {
  console.log("Welcome to " + school);
}
showSchool();
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment11\Task3> node
task3.js
Welcome to Sample College
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the difference between block scope and global scope?
Ans. A block variable works only inside its block, a global variable works everywhere.
Q2. What is a closure?
Ans. It is a function that remembers the variables of the place where it was created.
Q3. Why does the counter keep its value between calls?
Ans. The returned function remembers the variable count.

Q4. What is shadowing?
Ans. A variable inside a block with the same name hides the outer variable in that block.

EXPERIMENT NO. 12

Date of Performance:- Course: BCS354

1. Experiment Title:

- Callbacks and Higher-Order Functions

2. Aim:

- To pass a function as an argument to another function and call it.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- In JavaScript a function is a value, so it can be passed to another function as an argument. The function that is passed is called a callback, and a function that takes or returns another function is called a higher-order function. Callbacks let the same function do different work.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: Function as an argument

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 12 | Task 1
function applyTwice(fn, value) {
  const result = fn(value);
  return fn(result);
}

function addTen(n) {
  return n + 10;
}

console.log(applyTwice(addTen, 5));
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment12\Task1> node
task1.js
25
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment12\Task1>
```

Task 2: Callback with an arrow function

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 12 | Task 2
function formatMarks(marks, callback) {
  return callback(marks);
}

console.log(formatMarks(76, (m) => "Marks: " + m + "/100"));
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment12\Task2> node
task2.js
Marks: 76/100
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment12\Task2>
```

Task 3: Repeat an action

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 12 | Task 3
function repeatAction(times, action) {
  for (let round = 1; round <= times; round++) {
    action(round);
  }
}
repeatAction(3, (round) => console.log("Round " + round));
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment12\Task3> node
task3.js
Round 1
Round 2
Round 3
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment12\Task3>
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is a callback function?

Ans. It is a function passed to another function so that it can be called later.

Q2. What is a higher-order function?

Ans. A function that takes or returns another function, for example forEach, map and filter.

Q3. Can a function be stored in a variable?

Ans. Yes, in JavaScript a function is a value.

Q4. What is the difference between fn and fn()?
Ans. fn is the function itself, fn() calls it.

EXPERIMENT NO. 13

Date of Performance:- Course: BCS354

1. Experiment Title:

- Reduce, Some, Every and Map

2. Aim:

- To use reduce, some, every and map on arrays, and to keep the original array unchanged.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- The reduce() method combines all items of an array into one value, starting from a given value such as 0. The some() method checks whether at least one item passes a test, and every() checks whether all items pass it. The map() method makes a new array by changing every item, and the original array stays the same.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: reduce

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 13 | Task 1
const prices = [120, 45, 300];
const total = prices.reduce((sum, price) => sum + price, 0);

console.log(total);
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment13\Task1> node
task1.js
465
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment13\Task1>
```

Task 2: some and every

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 13 | Task 2
const marks = [72, 85, 30, 90];

console.log("Any mark below 40: " + marks.some((m) => m < 40));
console.log("All marks 35 or more: " + marks.every((m) => m >= 35));
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment13\Task2> node
task2.js
Any mark below 40: true
All marks 35 or more: false
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment13\Task2>
```

Task 3: map

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 13 | Task 3
const marks = [72, 85, 64, 90];
const bonus = marks.map((mark) => mark + 5);

console.log(bonus);
console.log(marks);
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment13\Task3> node
task3.js
[ 77, 90, 69, 95 ]
[ 72, 85, 64, 90 ]
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment13\Task3>
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What does reduce() do?

Ans. It combines all items of an array into one value.

Q2. What is the difference between some() and every()?

Ans. some() is true if at least one item passes, every() is true only if all items pass.

Q3. Does map() change the original array?

Ans. No, it returns a new array.

Q4. Why do we give 0 as the starting value when adding?

Ans. So that the total starts from zero.

EXPERIMENT NO. 14

Date of Performance:- Course: BCS354

1. Experiment Title:

- Timers and Intervals

2. Aim:

- To run code after a delay with setTimeout, to cancel a timer, and to repeat code with setInterval.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- The setTimeout() function runs a function once after a delay given in milliseconds, and the program does not wait for it. The clearTimeout() function cancels a timer before it runs.
- The setInterval() function runs a function again and again after every delay until clearInterval() stops it.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: setTimeout

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 14 | Task 1
console.log("Start");

setTimeout(() => {
  console.log("Timer done");
}, 0);

console.log("End");
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment14\Task1> node
task1.js
Start
End
Timer done
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment14\Task1>
```

Task 2: clearTimeout

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 14 | Task 2
const timerId = setTimeout(() => {
  console.log("Not shown");
}, 1000);

clearTimeout(timerId);
console.log("Cancelled");
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment14\Task2> node
task2.js
Cancelled
```

Task 3: setInterval and clearInterval

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 14 | Task 3
let count = 0;

const id = setInterval(() => {
  count++;
  console.log("Tick " + count);
  if (count === 3) {
    clearInterval(id);
  }
}, 200);
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment14\Task3> node
task3.js
Tick 1
Tick 2
Tick 3
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is the difference between setTimeout and setInterval?
Ans. setTimeout runs once after a delay, setInterval keeps running after every delay.
Q2. Why does Timer done print after End even with delay 0?
Ans. The timer function runs only after the current code has finished.
Q3. How do you cancel a timer?
Ans. With clearTimeout and the id returned by setTimeout.

Q4. In which unit is the delay given?
Ans. In milliseconds.

EXPERIMENT NO. 15

Date of Performance:- Course: BCS354

1. Experiment Title:

- Promises and async/await with Local Data

2. Aim:

- To create a Promise that resolves or rejects, to use async/await with try and catch, and to load local data as a Promise.

3. Software / Tools Required:

- Visual Studio Code, Node.js, Google Chrome

4. Theory:

- A Promise represents a value that will be available later. It is resolved with resolve() when the work succeeds and rejected with reject() when it fails. An async function can use await to wait for a Promise, and try and catch handles a rejected Promise.

5. Algorithm / Procedure:

1. Open Visual Studio Code.
2. Create a new file with the .js or .html extension.
3. Write the program and save the file.
4. Run the .js file with Node.js, or open the .html file in the browser.
5. Check the output.

Task 1: Promise that resolves

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 15 | Task 1
function checkMarks(marks) {
  return new Promise((resolve, reject) => {
    if (marks >= 0 && marks <= 100) {
      resolve(marks);
    } else {
      reject(new Error("Marks out of range"));
    }
  });
}

checkMarks(80).then((value) => {
  console.log(value);
});
```

7. Input:

- Run the file using Node.js: node task1.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment15\Task1> node
task1.js
80
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment15\Task1>
```

Task 2: Promise that rejects

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 15 | Task 2
function checkMarks(marks) {
  return new Promise((resolve, reject) => {
    if (marks >= 0 && marks <= 100) {
      resolve(marks);
    } else {
      reject(new Error("Marks out of range"));
    }
  });
}

checkMarks(120).catch((error) => {
  console.log(error.message);
});
```

7. Input:

- Run the file using Node.js: node task2.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment15\Task2> node
task2.js
Marks out of range
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment15\Task2>
```

Task 3: async/await with try and catch

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 15 | Task 3
function checkMarks(marks) {
  return new Promise((resolve, reject) => {
    if (marks >= 0 && marks <= 100) {
      resolve(marks);
    } else {
      reject(new Error("Marks out of range"));
    }
  });
}

async function run() {
  try {
    console.log(await checkMarks(65));
    console.log(await checkMarks(-5));
  } catch (error) {
    console.log("Caught: " + error.message);
  }
}

run();
```

7. Input:

- Run the file using Node.js: node task3.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment15\Task3> node
task3.js
65
Caught: Marks out of range
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment15\Task3>
```

Task 4: Load local data

6. Program:

```
// Name: Shubhankar Singh | ID: BCS2025390 | Assignment 15 | Task 4
function getUsers() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve([
        { id: 1, name: "Asha" },
        { id: 2, name: "Rohan" },
        { id: 3, name: "Meera" }
      ]);
    }, 100);
  });
}

async function showUsers() {
  const users = await getUsers();
  users.forEach((user) => {
    console.log(user.name);
  });
}

showUsers();
```

7. Input:

- Run the file using Node.js: node task4.js

8. Output:

```
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment15\Task4> node
task4.js
Asha
Rohan
Meera
C:\Users\Shubhankar\Desktop\JAVA SCRIPT\js_lab\js_lab\Assignment15\Task4>
```

9. Result:

- The programs were written and executed successfully, and the expected output was obtained.

10. Viva-Voce Questions:

Q1. What is a Promise?
Ans. It is an object for a value that will be available later.
Q2. What is the difference between resolve and reject?
Ans. resolve completes the Promise successfully, reject completes it with an error.
Q3. What does await do?
Ans. It waits for a Promise to finish and gives its value.

Q4. Why do we use try and catch with await?
Ans. To handle the error when a Promise is rejected.